

Follow Recommendations Using Graph Mining

Graphs can be used to represent a wide range of phenomena. This is particularly true for online social networks, and the **Internet of Things (IoT)**. Graph mining is big business, with websites such as Facebook running on data analysis experiments performed on graphs.

Social media websites are built upon engagement. Users without active news feeds, or interesting friends to follow, do not engage with sites. In contrast, users with more interesting friends and *followees* engage more, see more ads. This leads to larger revenue streams for the website.

In this chapter, we look at how to define similarity on graphs, and how to use them within a data mining context. Again, this is based on a model of the phenomena. We look at some basic graph concepts, like sub-graphs and connected components. This leads to an investigation of cluster analysis, which we delve more deeply into in [Chapter 10](#), *Clustering News Articles*.

The topics covered in this chapter include:

- Clustering data to find patterns
- Loading datasets from previous experiments
- Getting follower information from Twitter
- Creating graphs and networks
- Finding subgraphs for cluster analysis

Loading the dataset

In this chapter, our task is to recommend users on online social networks based on shared connections. Our logic is that if two users have the same friends, they are highly similar and worth recommending to each other. We want our recommendations to be of high value. We can only recommend so many people before it becomes tedious, therefore we need to find recommendations that engage users.

To do this, we use the previous chapter's disambiguation model to find only users talking about *Python as a programming language*. In this chapter, we use the results from one data mining experiment as input into another data mining experiment. Once we have our Python programmers selected, we then use their friendships to find clusters of users that are highly similar to each other. The similarity between two users will be defined by how many friends they have in common. Our intuition will be that the more friends two people have in common, the more likely two people are to be friends (and therefore should be on our social media platform).

We are going to create a small social graph from Twitter using the API we introduced in the previous chapter. The data we are looking for is a subset of users interested in a similar topic (again, the Python programming language) and a list of all of their friends (people they follow). With this data, we will check how similar two users are, based on how many friends they have in common.



There are many other online social networks apart from Twitter. The reason we have chosen Twitter for this experiment is that their API makes it quite easy to get this sort of information. The information is available from other sites, such as Facebook, LinkedIn, and Instagram, as well. However, getting this information is more difficult.

To start collecting data, set up a new Jupyter Notebook and an instance of the twitter connection, as we did in the previous chapter. You can reuse the app information from the previous chapter or create a new one:

```
import twitter
consumer_key = "<Your Consumer Key Here>"
consumer_secret = "<Your Consumer Secret Here>"
access_token = "<Your Access Token Here>"
access_token_secret = "<Your Access Token Secret Here>"
authorization = twitter.OAuth(access_token,
access_token_secret, consumer_key, consumer_secret)
t = twitter.Twitter(auth=authorization, retry=True)
```

Also, set up the filenames. You will want to use a different folder for this experiment from the one you used in [Chapter 6, Social Media Insight Using Naive Bayes](#), ensuring you do not override your previous dataset!

```
import os
data_folder = os.path.join(os.path.expanduser("~"), "Data", "twitter")
output_filename = os.path.join(data_folder, "python_tweets.json")
```

Next, we will need a list of users. We will do a search for tweets, as we did in the previous chapter, and look for those mentioning the word python. First, create two lists for storing the tweet's text and the corresponding users. We will need the user IDs later, so we create a dictionary mapping that now. The code is as follows:

```
original_users = []  
tweets = []  
user_ids = {}
```

We will now perform a search for the word python, as we did in the previous chapter, and iterate over the search results and only saving Tweets with text (as per the last chapter's requirements):

```
search_results = t.search.tweets(q="python", count=100)['statuses']  
for tweet in search_results:  
    if 'text' in tweet:  
        original_users.append(tweet['user']['screen_name'])  
        user_ids[tweet['user']['screen_name']] = tweet['user']['id']  
        tweets.append(tweet['text'])
```

Running this code will get about 100 tweets, maybe a little fewer in some cases. Not all of them will be related to the programming language, though. We will address that by using the model we trained in the previous chapter.

Classifying with an existing model

As we learned in the previous chapter, not all tweets that mention the word python are going to be relating to the programming language. To do that, we will use the classifier we used in the previous chapter to get tweets based on the programming language. Our classifier wasn't perfect, but it will result in a better specialization than just doing the search alone.

In this case, we are only interested in users who are tweeting about Python, the programming language. We will use our classifier from the last chapter to determine which tweets are related to the programming language. From there, we will select only those users who were tweeting about the programming language.

To do this part of our broader experiment, we first need to save the model from the previous chapter. Open the Jupyter Notebook we made in the last chapter, the one in which we built and trained the classifier.



If you have closed it, then the Jupyter Notebook won't remember what you did, and you will need to run the cells again. To do this, click on the Cell menu on the Notebook and choose Run All.

After all of the cells have computed, choose the final blank cell. If your Notebook doesn't have a blank cell at the end, choose the last cell, select the Insert menu, and select the Insert Cell Below option.

We are going to use the `joblib` library to save our model and load it.



`joblib` is included with the `scikit-learn` package as a built-in external package. No extra installation step needed! This library has tools for saving and loading models, and also for simple parallel processing - which is used in `scikit-learn` quite a lot.

First, import the library and create an output filename for our model (make sure the directories exist, or else they won't be created). I've stored this model in my `Models` directory, but you could choose to store them somewhere else. The code is as follows:

```
from sklearn.externals import joblib
output_filename = os.path.join(os.path.expanduser("~"), "Models", "twitter", "python_context.pk"
```

Next, we use the `dump` function in `joblib`, which works much like the similarly named version in the `json` library. We pass the model itself and the output filename:

```
joblib.dump(model, output_filename)
```

Running this code will save our model to the given filename. Next, go back to the new Jupyter Notebook you created in the last subsection and load this model.

Using my data, this comes up to 46 relevant users. A little lower than our 100 tweets/users from before, but now we have a basis for building our social network. We can always add more data to get more users, but 40+ users will be sufficient to go through this chapter as a first pass. I recommend coming back, adding more data, and running the code again, to see what results you obtain.

You will need to set the model's filename again in this Notebook by copying the following code:

```
model_filename = os.path.join(os.path.expanduser("~"), "Models", "twitter", "python_context.pkl"
```

Make sure the filename is the one you used just before to save the model. Next, we need to recreate our BagOfWords class, as it was a custom-built class and can't be loaded directly by joblib. Simply copy the entire BagOfWords class from the previous chapter's code, including its dependencies:

```
import spacy
from sklearn.base import TransformerMixin

# Create a spaCy parser
nlp = spacy.load('en')

class BagOfWords(TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        results = []
        for document in X:
            row = {}
            for word in list(nlp(document, tag=False, parse=False, entity=False)):
                if len(word.text.strip()): # Ignore words that are just whitespace
                    row[word.text] = True
            results.append(row)
        return results
```



In production, you would need to develop your custom transformers in separate, centralized files, and import them into the Notebook instead. This little hack simplifies the workflow, but feel free to experiment with centralizing important code by creating a library of common functionality.

Loading the model now just requires a call to the `load` function of `joblib`:

```
from sklearn.externals import joblib
context_classifier = joblib.load(model_filename)
```

Our `context_classifier` works exactly like the model object of the notebook we saw in [Chapter 6, Social Media Insight Using Naive Bayes](#). It is an instance of a Pipeline, with the same three steps as before (BagOfWords, DictVectorizer, and a BernoulliNB classifier). Calling the `predict` function on this model gives us a prediction as to whether our tweets are relevant to the programming language. The code is as follows:

```
y_pred = context_classifier.predict(tweets)
```

The *i*th item in `y_pred` will be 1 if the *i*th tweet is (predicted to be) related to the programming language, or else it will be 0. From here, we can get just the tweets that are relevant and their relevant users:

```
relevant_tweets = [tweets[i] for i in range(len(tweets)) if y_pred[i] == 1]
relevant_users = [original_users[i] for i in range(len(tweets)) if y_pred[i] == 1]
```

Getting follower information from Twitter

With our initial set of users, we now need to get the friends of each of these users. A friend is a person whom the user is following. The API for this is called friends/ids, and it has both good and bad points. The good news is that it returns up to 5,000 friend IDs in a single API call. The bad news is that you can only make 15 calls every 15 minutes, which means it will take you at least 1 minute per user to get all followers—more if they have more than 5,000 friends (which happens more often than you may think).

The code is similar to the code from our previous API usage (obtaining tweets). We will package it as a function, as we will use this code in the next two sections. Our function takes a twitter user's ID value, and returns their friends. While it may be surprising to some, many Twitter users have more than 5,000 friends. Due to this we will need to use Twitter's pagination function, which lets Twitter return multiple pages of data through separate API calls. When you ask Twitter for information, it gives you your information along with a cursor, which is an integer that Twitter uses to track your request. If there is no more information, this cursor is 0; otherwise, you can use the supplied cursor to get the next page of results. Passing this cursor lets twitter continue your query, returning the next set of data to you.

In the function, we keep looping while this cursor is not equal to 0 (as, when it is, there is no more data to collect). We then perform a request for the user's followers and add them to our list. We do this in a try block, as there are possible errors that can happen that we can handle. The follower's IDs are stored in the ids key of the results dictionary. After obtaining that information, we update the cursor. It will be used in the next iteration of the loop. Finally, we check if we have more than 10,000 friends. If so, we break out of the loop. The code is as follows:

```
import time

def get_friends(t, user_id):
    friends = []
    cursor = -1
    while cursor != 0:
        try:
            results = t.friends.ids(user_id= user_id, cursor=cursor, count=5000)
            friends.extend([friend for friend in results['ids']])
            cursor = results['next_cursor']
            if len(friends) >= 10000:
                break
        except TypeError as e:
            if results is None:
                print("You probably reached your API limit, waiting for 5 minutes")
                sys.stdout.flush()
                time.sleep(5*60) # 5 minute wait
            else:
                # Some other error happened, so raise the error as normal
                raise e
        except twitter.TwitterHTTPError as e:
            print(e)
            break
    finally:
        # Break regardless -- this stops us going over our API limit
        time.sleep(60)
```



It is worth inserting a warning here. We are dealing with data from the Internet, which means weird



things can and do happen regularly. A problem I ran into when developing this code was that some users have many, many, many thousands of friends. As a fix for this issue, we will put a failsafe here, exiting the function if we reach more than 10,000 users. If you want to collect the full dataset, you can remove these lines, but beware that it may get stuck on a particular user for a very long time.

Much of the above function is error handling, as quite a lot can go wrong when dealing with external APIs!

The most likely error that can happen is if we accidentally reach our API limit (while we have a sleep to stop that, it can occur if you stop and run your code before this sleep finishes). In this case, results is None and our code will fail with a `TypeError`. In this case, we wait for 5 minutes and try again, hoping that we have reached our next 15-minute window. There may be another `TypeError` that occurs at this time. If one of them does, we raise it and will need to handle it separately.

The second error that can happen occurs at Twitter's end, such as asking for a user that doesn't exist or some other data-based error, resulting in a `TwitterHTTPError` (which is a similar concept to a HTTP 404 error). In this case, don't try this user anymore, and just return any followers we did get (which, in this case, is likely to be 0).

Finally, Twitter only lets us ask for follower information 15 times every 15 minutes, so we will wait for 1 minute before continuing. We do this in a finally block so that it happens even if an error occurs.

Building the network

Now we are going to build our network of users, where users are linked if the two users follow each other. The aim of building this network is to give us a data structure we can use to segment our list of users into groups. From these groups, we can then recommend people in the same group to each other. Starting with our original users, we will get the friends for each of them and store them in a dictionary. Using this concept we can grow the graph outwards from an initial set of users.

Starting with our original users, we will get the friends for each of them and store them in a dictionary (after obtaining the user's ID from our *user_id* dictionary):

```
friends = {}
for screen_name in relevant_users:
    user_id = user_ids[screen_name]
    friends[user_id] = get_friends(t, user_id)
```

Next, we are going to remove any user who doesn't have any friends. For these users, we can't really make a recommendation in this way. Instead, we might have to look at their content or people who follow them. We will leave that out of the scope of this chapter, though, so let's just remove these users. The code is as follows:

```
friends = {user_id:friends[user_id]
            for user_id in friends
            if len(friends[user_id]) > 0}
```

We now have between 30 and 50 users, depending on your initial search results. We are now going to increase that amount to 150. The following code will take quite a long time to run—given the limits on the API, we can only get the friends for a user once every minute. Simple math will tell us that 150 users will take 150 minutes, which is at least 2 hours and 30 minutes. Given the time we are going to be spending on getting this data, it pays to ensure we get only good users.

What makes a good user, though? Given that we will be looking to make recommendations based on shared connections, we will search for users based on shared connections. We will get the friends of our existing users, starting with those users who are better connected to our existing users. To do that, we maintain a count of all the times a user is in one of our friend's lists. It is worth considering the goals of the application when considering your sampling strategy. For this purpose, getting lots of similar users enables the recommendations to be more regularly applicable.

To do this, we simply iterate over all the friend lists we have and then count each time a friend occurs.

```
from collections import defaultdict
def count_friends(friends):
    friend_count = defaultdict(int)
    for friend_list in friends.values():
        for friend in friend_list:
            friend_count[friend] += 1
    return friend_count
```

Computing our current friend count, we can then get the most connected (that is, most friends from our existing list) person from our sample. The code is as follows:

```
friend_count = count_friends(friends)
from operator import itemgetter
best_friends = sorted(friend_count, key=friend_count.get, reverse=True)
```

From here, we set up a loop that continues until we have the friends of 150 users. We then iterate over all of our best friends (which happens in order of the number of people who have them as friends) until we find a user we have not yet checked. We then get the friends of that user and update the friends counts. Finally, we work out who is the most connected user who we haven't already got in our list:

```
while len(friends) < 150:
    for user_id, count in best_friends:
        if user_id in friends:
            # Already have this user, move to next one
            continue
        friends[user_id] = get_friends(t, user_id)
        for friend in friends[user_id]:
            friend_count[friend] += 1
        best_friends = sorted(friend_count.items(), key=itemgetter(1), reverse=True)
        break
```

The codes will then loop and continue until we reach 150 users.



You may want to set these values lower, such as 40 or 50 users (or even just skip this bit of code temporarily). Then, complete the chapter's code and get a feel for how the results work. After that, reset the number of users in this loop to 150, leave the code to run for a few hours, and then come back and rerun the later code.

Given that collecting that data probably took nearly 3 hours, it would be a good idea to save it in case we have to turn our computer off. Using the json library, we can easily save our friends dictionary to a file:

```
import json
friends_filename = os.path.join(data_folder, "python_friends.json")
with open(friends_filename, 'w') as outf:
    json.dump(friends, outf)
```

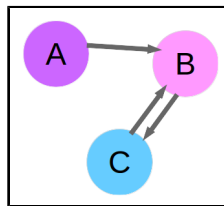
If you need to load the file, use the json.load function:

```
with open(friends_filename) as inf:
    friends = json.load(inf)
```

Creating a graph

At this point in our experiment, we have a list of users and their friends. This gives us a graph where some users are friends of other users (although not necessarily the other way around).

A **graph** is a set of nodes and edges. Nodes are usually objects of interest - in this case, they are our users. The edges in this initial graph indicate that user A is a friend of user B. We call this a **directed graph**, as the order of the nodes matters. Just because user A is a friend of user B, that doesn't imply that user B is a friend of user A. The example network below shows this, along with a user C who is friends of user B, and is friended in turn by user B as well:



In python, one of the best libraries for working with graphs, including creating, visualising and computing, is called **NetworkX**.



Once again, you can use Anaconda to install NetworkX: `conda install networkx`

First, we create a directed graph using NetworkX. By convention, when importing NetworkX, we use the abbreviation `nx` (although this isn't necessary). The code is as follows:

```
import networkx as nx
G = nx.DiGraph()
```

We will only visualize our key users, not all of the friends (as there are many thousands of these and it is hard to visualize). We get our main users and then add them to our graph as nodes:

```
main_users = friends.keys()
G.add_nodes_from(main_users)
```

Next we set up the edges. We create an edge from a user to another user if the second user is a friend of the first user. To do this, we iterate through all of the friends of a given user. We ensure that the friend is one of our main users (as we currently aren't interested in visualizing the other users), and add the edge if they are.

```
for user_id in friends:
    for friend in friends[user_id]:
```

```

if str(friend) in main_users:
    G.add_edge(user_id, friend)

```

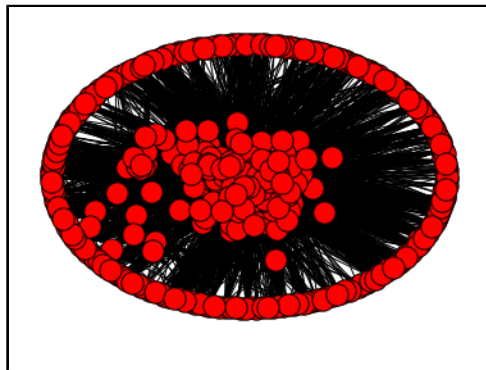
We can now visualize the network using NetworkX's draw function, which uses matplotlib. To get the image in our notebook, we use the inline function on matplotlib and then call the draw function. The code is as follows:

```

%matplotlib inline
nx.draw(G)

```

The results are a bit hard to make sense of; they show just the ring of nodes, and its hard to work anything specific out about the dataset. Not a good image at all:



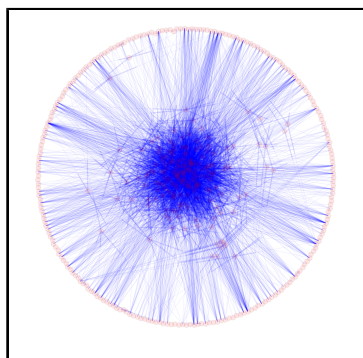
We can make the graph a bit better by using pyplot to handle the creation of the figure, which is used by NetworkX to do graph drawing. Import pyplot, create a larger figure, and then call NetworkX's draw function to increase the size of the image:

```

from matplotlib import pyplot as plt
plt.figure(3,figsize=(20,20))
nx.draw(G, alpha=0.1, edge_color='b')

```

By making the graph bigger and adding transparency, an outline of how the graph appears can now be seen:



In my graph, there was a major group of users all highly connected to each other, and most other users didn't have many connections at all. As you can see, it is very well connected in the center!

This is actually a property of our method of choosing new users—we choose those who are already well linked in our graph, so it is likely they will just make this group larger. For social networks, generally the number of connections a user has follows a power law. A small percentage of users have many connections, and others have only a few. The shape of the graph is often described as having a *long tail*.

By zooming into parts of the graph you can start seeing structure. Visualizing and analyzing graphs like this is hard - we will see some tools for making this process easier in the next section.

Creating a similarity graph

The final step to this experiment is to recommend users, based on how many friends they share. As mentioned previously, our logic is that, if two users have the same friends, they are highly similar. We could recommend one user to the other on this basis.

We are therefore going to take our existing graph (which has edges relating to friendship) and create a new graph from its information. The nodes are still users, but the edges are going to be **weighted edges**. A weighted edge is simply an edge with a weight property. The logic is that a higher weight indicates more similarity between the two nodes than a lower weight. This is context-dependent. If the weights represent distance, then the lower weights indicate more similarity.

For our application, the weight will be the similarity of the two users connected by that edge (based on the number of friends they share). This graph also has the property that it is not directed. This is due to our similarity computation, where the similarity of user A to user B is the same as the similarity of user B to user A.



Other similarity measurements are directed. An example is ratio of similar users, which is the number of friends in common divided by the user's total number of friends. In this case, you would need a directed graph.

There are many ways to compute the similarity between two lists like this. For example, we could compute the number of friends the two have in common. However, this measure is always going to be higher for people with more friends. Instead, we can normalize it by dividing by the total number of distinct friends the two have. This is called the **Jaccard Similarity**.

The Jaccard Similarity, always between 0 and 1, represents the percentage overlap of the two. As we saw in Chapter 2, *Classifying with scikit-learn Estimators*, normalization is an important part of data mining exercises and generally a good thing to do. There are fringe cases where you wouldn't normalize data, but by default normalize first.

To compute the Jaccard similarity, we divide the intersection of the two sets of followers by the union of the two. These are set operations and we have lists, so we will need to convert the friends lists to sets first. The code is as follows:

```
| friends = {user: set(friends[user]) for user in friends}
```

We then create a function that computes the similarity of two sets of friends lists. The code is as follows:

```
| def compute_similarity(friends1, friends2):  
|     return len(friends1 & friends2) / (len(friends1 | friends2) + 1e-6)
```



We add 1e-6 (or 0.000001) to the similarity above to ensure we never get a division by zero error, in cases where neither user has any friends. It is small enough to not really affect our results, but big enough to be more than zero.

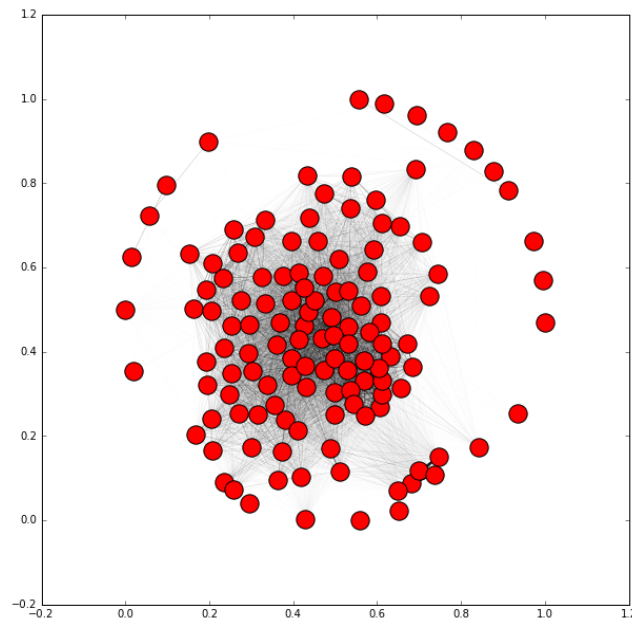
Next, we draw the edges. To get the weights, we iterate over the edges in the graph (in a specific order) and collect the weights:

```
| edgewidth = [ d['weight'] for (u,v,d) in G.edges(data=True)]
```

We then draw the edges:

```
| nx.draw_networkx_edges(G, pos, width=edgewidth)
```

The result will depend on your data, but it will typically show a graph with a large set of nodes connected quite strongly and a few nodes poorly connected to the rest of the network.



The difference in this graph compared to the previous graph is that the edges determine the similarity between the nodes based on our similarity metric and not on whether one is a friend of another (although there are similarities between the two!). We can now start extracting information from this graph in order to make our recommendations.

From here, we can create our weighted graph of the similarity between users. We will use this quite a lot in the rest of the chapter, so we will create a function to perform this action. Let's take a look at the threshold parameter:

```
def create_graph(followers, threshold=0):
    G = nx.Graph()
    for user1 in friends.keys():
        for user2 in friends.keys():
            if user1 == user2:
                continue
            weight = compute_similarity(friends[user1], friends[user2])
            if weight >= threshold:
                G.add_node(user1)
                G.add_node(user2)
                G.add_edge(user1, user2, weight=weight)
    return G
```

We can now create a graph by calling this function. We start with no threshold, which means all links are created. The code is as follows:

```
G = create_graph(friends)
```

The result is a very strongly connected graph—all nodes have edges, although many of those will have a weight of 0. We will see the weight of the edges by drawing the graph with line widths relative to the weight of the edge—thicker lines indicate higher weights.

Due to the number of nodes, it makes sense to make the figure larger to get a clearer sense of the connections:

```
plt.figure(figsize=(10,10))
```

We are going to draw the edges with a weight, so we need to draw the nodes first. NetworkX uses layouts to determine where to put the nodes and edges, based on certain criteria. Visualizing networks is a very difficult problem, especially as the number of nodes grows. Various techniques exist for visualizing networks, but the degree to which they work depends heavily on your dataset, personal preferences, and the aim of the visualization. I found that the `spring_layout` worked quite well, but other options such as `circular_layout` (which is a good default if nothing else works), `random_layout`, `shell_layout`, and `spectral_layout` also exist and have uses where the others may fail.



Visit <http://networkx.lanl.gov/reference/drawing.html> for more details on layouts in NetworkX. Although it adds some complexity, the `draw_graphviz` option works quite well and is worth investigating for better visualizations. It is well worth considering in real-world uses.

Let's use `spring_layout` for visualization:

```
pos = nx.spring_layout(G)
```

Using our `pos` layout, we can then position the nodes:

```
nx.draw_networkx_nodes(G, pos)
```


Finding subgraphs

From our similarity function, we could simply rank the results for each user, returning the most similar user as a recommendation - as we did with our product recommendations. This works, and is indeed one way to perform this type of analysis.

Instead, we might want to find clusters of users that are all similar to each other. We could advise these users to start a group, create advertising targeting this segment, or even just use those clusters to do the recommendations themselves. Finding these clusters of similar users is a task called **cluster analysis**.



Cluster analysis is a difficult task, with complications that classification tasks do not typically have. For example, evaluating classification results is relatively easy - we compare our results to the ground truth (from our training set) and see what percentage we got right. With cluster analysis, though, there isn't typically a ground truth. Evaluation usually comes down to seeing if the clusters make sense, based on some preconceived notion we have of what the cluster should look like.

Another complication with cluster analysis is that the model can't be trained against the expected result to learn—it has to use some approximation based on a mathematical model of a cluster, not what the user is hoping to achieve from the analysis.

Due to these issues, cluster analysis is more of an exploratory tool, rather than a prediction tool. Some research and applications use clustering for analysis, but its usefulness as a predictive model is dependent on an analyst selecting parameters and finding graphs that *look right*, rather than a specific evaluation metric.

Connected components

One of the simplest methods for clustering is to find the **connected components** in a graph. A connected component is a set of nodes in a graph that are connected via edges. Not all nodes need to be connected to each other to be a connected component. However, for two nodes to be in the same connected component, there needs to be a way to *travel* from one node to another in that connected component by moving along edges.



Connected components do not consider edge weights when being computed; they only check for the presence of an edge. For that reason, the code that follows will remove any edge with a low weight.

NetworkX has a function for computing connected components that we can call on our graph. First, we create a new graph using our `create_graph` function, but this time we pass a threshold of 0.1 to get only those edges that have a weight of at least 0.1, indicative of 10% of followers in common between the two node users:

```
G = create_graph(friends, 0.1)
```

We then use NetworkX to find the connected components in the graph:

```
sub_graphs = nx.connected_component_subgraphs(G)
```

To get a sense of the sizes of the graph, we can iterate over the groups and print out some basic information:

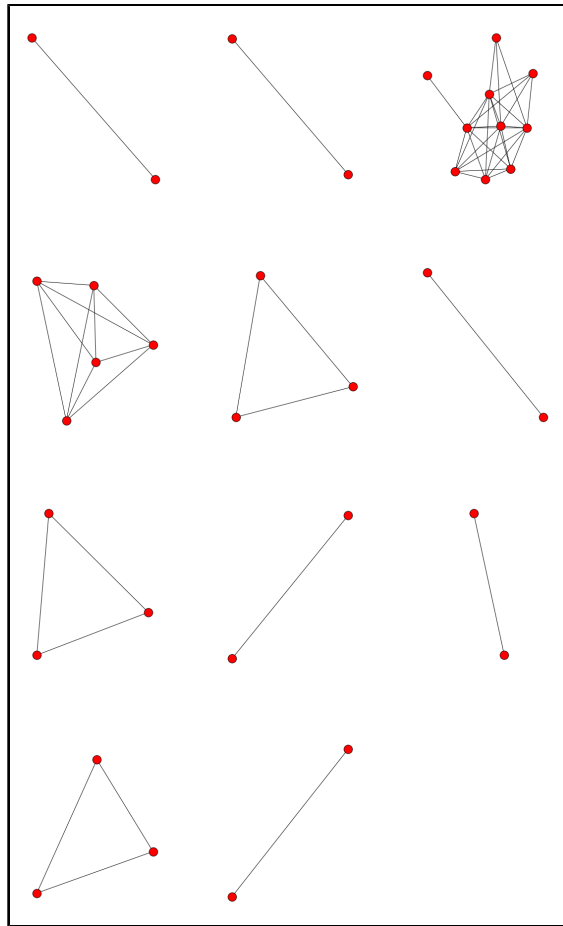
```
for i, sub_graph in enumerate(sub_graphs):
    n_nodes = len(sub_graph.nodes())
    print("Subgraph {0} has {1} nodes".format(i, n_nodes))
```

The results will tell you how big each of the connected components is. My results had one large subgraph of 62 users and lots of little ones with a dozen or fewer users.

We can alter the **threshold** to alter the connected components. This is because a higher threshold has fewer edges connecting nodes, and therefore will have smaller connected components and more of them. We can see this by running the preceding code with a higher threshold:

```
G = create_graph(friends, 0.25)
sub_graphs = nx.connected_component_subgraphs(G)
for i, sub_graph in enumerate(sub_graphs):
    n_nodes = len(sub_graph.nodes())
    print("Subgraph {0} has {1} nodes".format(i, n_nodes))
```

The preceding code gives us much smaller subgraphs and more of them. My largest cluster was broken into at least three parts and none of the clusters had more than 10 users. An example cluster is shown in the following figure, and



If you are not seeing anything on your graphs, try rerunning the line:

`sub_graphs = nx.connected_component_subgraphs(G)`

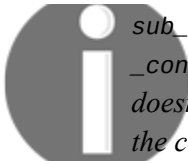
The `sub_graphs` object is a generator and is "consumed" after being used.

the connections within this cluster are also shown. Note that, as it is a connected component, there were no edges from nodes in this component to other nodes in the graph (at least, with the threshold set at 0.25).

We can draw the entire graph, showing each connected component in a different color. As these connected components are not connected to each other, it actually makes little sense to plot these on a single graph. This is because the positioning of the nodes and components is arbitrary, and it can confuse the visualization. Instead, we can plot each separately on a separate subfigure.

In a new cell, obtain the connected components and also the count of the connected components:

```
sub_graphs = nx.connected_component_subgraphs(G)
n_subgraphs = nx.number_connected_components(G)
```



sub_graphs is a generator, not a list of the connected components. For this reason, use `nx.number_connected_components` to find out how many connected components there are; don't use `len`, as it doesn't work due to the way that NetworkX stores this information. This is why we need to recompute the connected components here.

Create a new pyplot figure and give enough room to show all of our connected components. For this reason, we allow the graph to increase in size with the number of connected components.

Next, iterate over each connected component and add a subplot for each. The parameters to `add_subplot` are the number of rows of subplots, the number of columns, and the index of the subplot we are interested in. My visualization uses three columns, but you can try other values instead of three (just remember to change both values):

```
fig = plt.figure(figsize=(20, (n_subgraphs * 3)))
for i, sub_graph in enumerate(sub_graphs):
    ax = fig.add_subplot(int(n_subgraphs / 3) + 1, 3, i + 1)
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    pos = nx.spring_layout(G)
    nx.draw_networkx_nodes(G, pos, sub_graph.nodes(), ax=ax, node_size=500)
    nx.draw_networkx_edges(G, pos, sub_graph.edges(), ax=ax)
```

The results visualize each connected component, giving us a sense of the number of nodes in each and also how connected they are.

Optimizing criteria

Our algorithm for finding these connected components relies on the **threshold** parameter, which dictates whether edges are added to the graph or not. In turn, this directly dictates how many connected components we discover and how big they are. From here, we probably want to settle on some notion of which is the *best* threshold to use. This is a very subjective problem, and there is no definitive answer. This is a major problem with any cluster analysis task.

We can, however, determine what we think a good solution should look like and define a metric based on that idea. As a general rule, we usually want a solution where:

- Samples in the same cluster (connected components) are highly *similar* to each other
- Samples in different clusters are highly *dissimilar* to each other

The **Silhouette Coefficient** is a metric that quantifies these points. Given a single sample, we define the Silhouette Coefficient as follows:

$$s = \frac{b - a}{\max(a, b)}$$

Where a is the **intra-cluster distance** or the average distance to the other samples in the sample's cluster, and b is the **inter-cluster distance** or the average distance to the other samples in the *next-nearest* cluster.



To compute the overall Silhouette Coefficient, we take the mean of the Silhouette Coefficients for each sample. A clustering that provides a Silhouette Coefficient close to the maximum of 1 has clusters that have samples all similar to each other, and these clusters are very spread apart. Values near 0 indicate that the clusters all overlap and there is little distinction between clusters. Values close to the minimum of -1 indicate that samples are probably in the wrong cluster, that is, they would be better off in other clusters.

Using this metric, we want to find a solution (that is, a value for the threshold) that maximizes the Silhouette Coefficient by altering the threshold parameter. To do that, we create a function that takes the threshold as a parameter and computes the Silhouette Coefficient.

We then pass this into the **optimize** module of SciPy, which contains the `minimize` function that is used to find the minimum value of a function by altering one of the parameters. While we are interested in maximizing the Silhouette Coefficient, SciPy doesn't have a `maximize` function. Instead, we minimize the inverse of the Silhouette (which is basically the same thing).

The scikit-learn library has a function for computing the Silhouette Coefficient, `sklearn.metrics.silhouette_score`; however, it doesn't fix the function format that is required by the SciPy `minimize` function. The `minimize` function requires the variable parameter to be first (in our case, the threshold value), and any arguments to be after it. In our

case, we need to pass the friends dictionary as an argument in order to compute the graph.



The Silhouette Coefficient is not defined unless there are at least two nodes (in order for distance to be computed at all). In this case, we define the problem scope as invalid. There are a few ways to handle this, but the easiest is to return a very poor score. In our case, the minimum value that the Silhouette Coefficient can take is -1, and we will return -99 to indicate an invalid problem. Any valid solution will score higher than this.

The function below incorporates all of these issues giving us a function that takes a threshold value and a friends list, and computes the Silhouette Coefficient. It does this by building a matrix from the graph using NetworkX's `to_scipy_sparse_matrix` function.

```
import numpy as np
from sklearn.metrics import silhouette_score

def compute_silhouette(threshold, friends):
    G = create_graph(friends, threshold=threshold)
    if len(G.nodes()) < 2:
        return -99
    sub_graphs = nx.connected_component_subgraphs(G)

    if not (2 <= nx.number_connected_components() < len(G.nodes()) - 1):
        return -99

    label_dict = {}
    for i, sub_graph in enumerate(sub_graphs):
        for node in sub_graph.nodes():
            label_dict[node] = i

    labels = np.array([label_dict[node] for node in G.nodes()])
    X = nx.to_scipy_sparse_matrix(G).todense()
    X = 1 - X
    return silhouette_score(X, labels, metric='precomputed')
```



For evaluating sparse datasets, I recommend that you look into V-Measure or Adjusted Mutual Information. These are both implemented in scikit-learn, but they have very different parameters for performing their evaluation.

The Silhouette Coefficient implementation in scikit-learn, at the time of writing, doesn't support sparse matrices. For this reason, we need to call the `todense` function. Typically, this is a bad idea--sparse matrices are usually used because the data typically shouldn't be in a dense format. In this case, it will be fine because our dataset is relatively small; however, don't try this for larger datasets.



We have two forms of inversion happening here. The first is taking the inverse of the similarity to compute a distance function; this is needed, as the Silhouette Coefficient only accepts distances. The second is the inverting of the Silhouette Coefficient score so that we can minimize with SciPy's optimize module.

Finally, we create the function that we will minimize. This function is the inverse of the `compute_silhouette` function, because we want lower scores to be better. We could do this in our `compute_silhouette` function--I've separated them here to clarify the different steps involved.

```
def inverted_silhouette(threshold, friends):
    return -compute_silhouette(threshold, friends)
```

This function creates a new function from an original function. When the new function is called, all of the same arguments and keywords are passed onto the original function and the return value is returned, except that this returned value is negated before it is returned.

Now we can do our actual optimization. We call the minimize function on the inverted compute_silhouette function we defined:

```
from scipy.optimize import minimize
result = minimize(inverted_silhouette, 0.1, args=(friends,))
```



This function will take quite a while to run. Our graph creation function isn't that fast, nor is the function that computes the Silhouette Coefficient. Decreasing the maxiter parameter's value will result in fewer iterations being performed, but we run the risk of finding a suboptimal solution.

Running this function, I got a threshold of 0.135 that returns 10 components. The score returned by the minimize function was -0.192. However, we must remember that we negated this value. This means our score was actually 0.192. The value is positive, which indicates that the clusters tend to be better separated than not (a good thing). We could run other models and check whether it results in a better score, which means that the clusters are better separated.

We could use this result to recommend users—if a user is in a specific connected component, then we can recommend other users in that same component. This recommendation follows our use of the Jaccard Similarity to find good connections between users, our use of connected components to split them up into clusters, and our use of the optimization technique to find the best model in this setting.

However, a large number of users may not be connected at all, so we will use a different algorithm to find clusters for them. We will see other methods for cluster analysis in [Chapter 10, Clustering News Articles](#).

Summary

In this chapter, we looked at graphs from social networks and how to do cluster analysis on them. We also looked at saving and loading models from scikit-learn by using the classification model we created in [Chapter 6, Social Media Insight Using Naive Bayes](#).

We created a graph of friends from the social network Twitter. We then examined how similar two users were, based on their friends. Users with more friends in common were considered more similar, although we normalize this by considering the overall number of friends they have. This is a commonly used way to infer knowledge (such as age or general topic of discussion) based on similar users. We can use this logic for recommending users to others—if they follow user X and user Y is similar to user X, they will probably like user Y. This is, in many ways, similar to our transaction-led similarity of previous chapters.

The aim of this analysis was to recommend users, and our use of cluster analysis allowed us to find clusters of similar users. To do this, we found connected components on a weighted graph we created based on this similarity metric. We used the NetworkX package for creating graphs, using our graphs, and finding these connected components.

We then used the Silhouette Coefficient, which is a metric that evaluates how good a clustering solution is. Higher scores indicate a better clustering, according to the concepts of intracluster and intercluster distance. SciPy's optimize module was used to find the solution that maximizes this value.

In this chapter, we saw a few opposites in action. Similarity is a measure between two objects, where higher values indicate more similarity between those objects. In contrast, distance is a measure where lower values indicate more similarity. Another contrast we saw was a loss function, where lower scores are considered better (that is, we lost less). Its opposite is the score function, where higher scores are considered better.

To extend the work in this chapter, examine the V-measure and Adjusted Mutual Information scores in scikit-learn. These replace the Silhouette Coefficient used in this chapter. Are the clusters that result from maximizing these metrics better than the Silhouette Coefficient's clusters? Further, how can you tell? Often, the problem with cluster analysis is that you cannot objectively tell and may use human intervention to choose the best option.

In the next chapter, we will see how to extract features from another new type of data—images. We will discuss how to use neural networks to identify numbers in images and develop a program to automatically beat CAPTCHA images.